MITTUNIVERSITETET

Laboration: System administration 2

Laborant: Hussein Alshaikhshaker

Kurs: DT149G

Datum: 2025-11-16

Innehållsförteckning

Processhantering 3
Prioritetsvärde & Niceness 3
Bakgrundsprocesser & Front 3
Schemaläggning 4
Crontab 6
System statistik 6
Vmstat 6
Processer 6
Memory 7
Swap 7
I/O 7
System 7
CPU 7
top 8
Minne & Swap 8
Last 9
W 9
Uptime 9
Skillnader 10
Logg filer 10
Dmesg 10
Syslog 11
Skillnader 11
Referenser 12

Processhantering

Prioritetsvärde & Niceness

Prioritetsvärdet är ett värde som operativsystemet tar hänsyn till. Varje process har ett prioritetsvärde där ju lägre värdet är, desto mer prioriterad process det är. 0 är det lästa lägsta värdet, tyder på väldigt viktig process som får gå längst fram i kön, alltså mer resurser som cpu-tid, minne osv.

Mitt skript som loopar oändligt fick process-id 1043, prioritetsvärde 20 (default värde), och niceness värdet 0 (också default).

Efter att jag startade om skriptet med lägre prioritetsnivå, med kommandot [nice -n 10 ./infiniteloop.sh], skriptet startades som en helt ny process med nytt process-id. Processen fick prioritetsvärde 30 och niceness 10, dvs att processen är snällare och därmed mindre viktig i operativsystemet perspektiv. Andra processer får gå före i kön.

Det är inte möjligt att sänka prioritetsvärdet utan sudo, däremot kan man sänka prioritetsvärdet till den lägsta nivån som går att nå, alltså 0. Detta nåddes genom kommandot [renice], som ändrar niceness värdet medan processen exekverar. Priortetsvärde = 20 + nicenessvärde. Så niceness värdet sattes till -20.

Bakgrundsprocesser & Front

[$./infiniteloop.sh &]

Detta startade processen infiniteloop.sh i bakgrunden. & tecknet gör att processer startar i bakgrunden till skillnad från att starta processen utan & tecknet då processen körs i förgrunden. Dock stängs den ned om man stänger av terminalen den startades med (förälderprocessen).

[$nohup ./infiniteloop.sh &]

Detta startar också processen i bakgrunden (i och med att man fortfarande tar med & tecknet) och talar för den att ignorera alla SIGHUP signaler den får. Kommandot nohub gör att processen explicit ignorerar SIGHUP signalen från föräldern. Nohup är ett väldigt användbart kommando som fortsätter köra processen även efter utloggning från systemet, kan vara bra för att köra program som till exempel behöver köras över natten, eller program som jobbar med stora databaser där man kanske vill/behöver logga ut från systemet.

bg %[jobnr]

Fortsätter exekvera koden i bakgrunden

fg %[jobnr]

Fortsätter exekvera koden i förgrunden

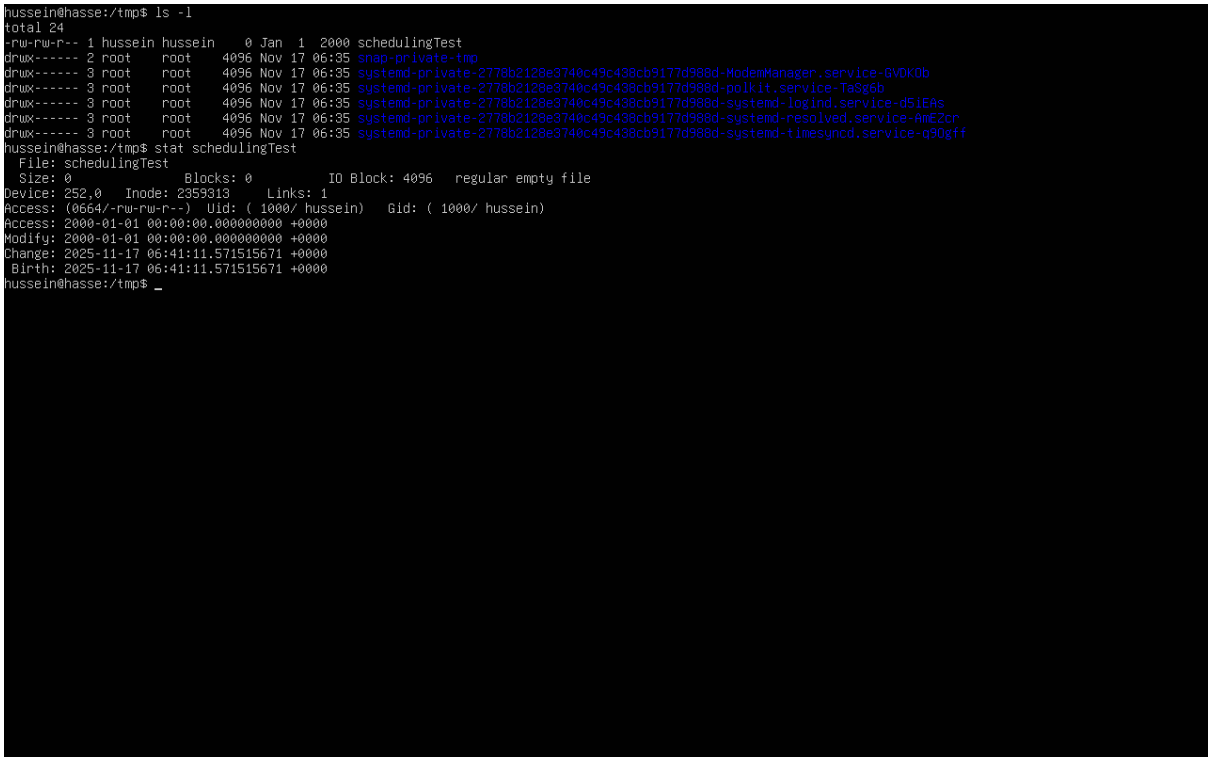
Schemaläggning

Inför denna uppgift behövde jag skapa en fiktiv fil och sätta senast ändrad datum mer än två dagar sedan.

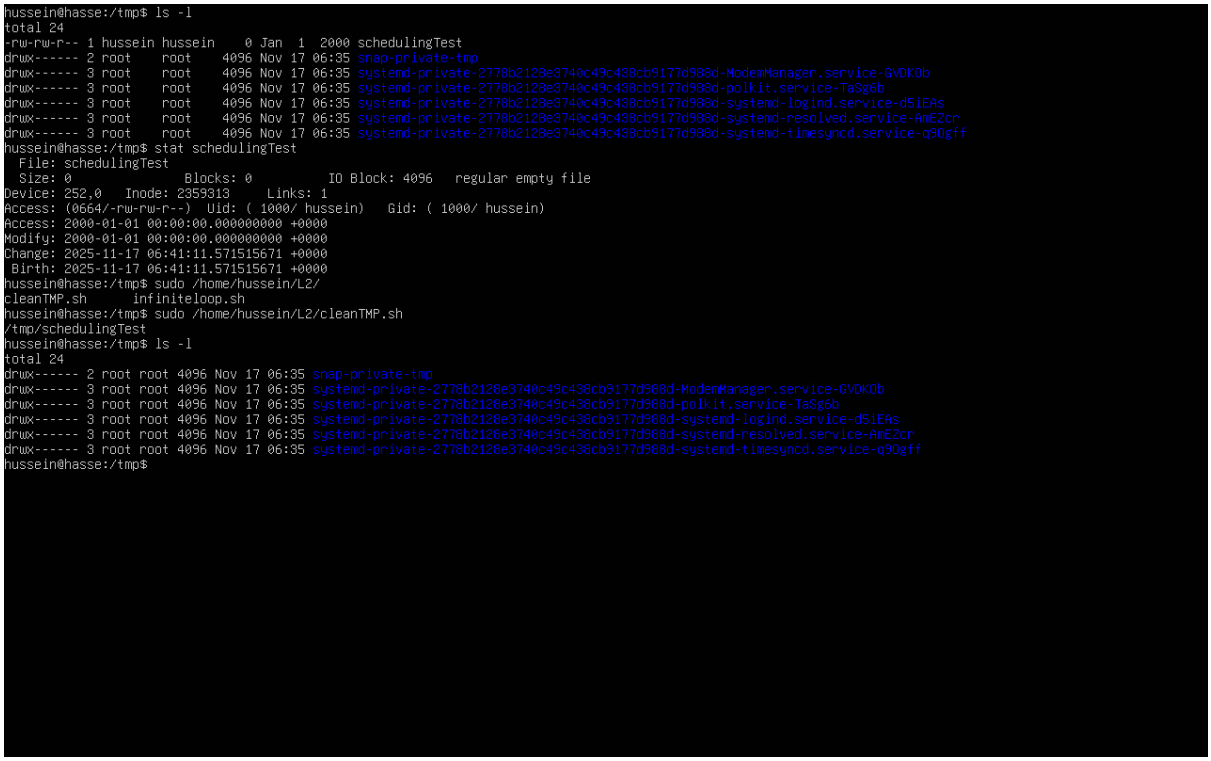
Nedan är mitt skript som rensar filer i /tmp mappen som inte har ändrats senaste 2 dagarna.



Figur 1: cleanTMP skript



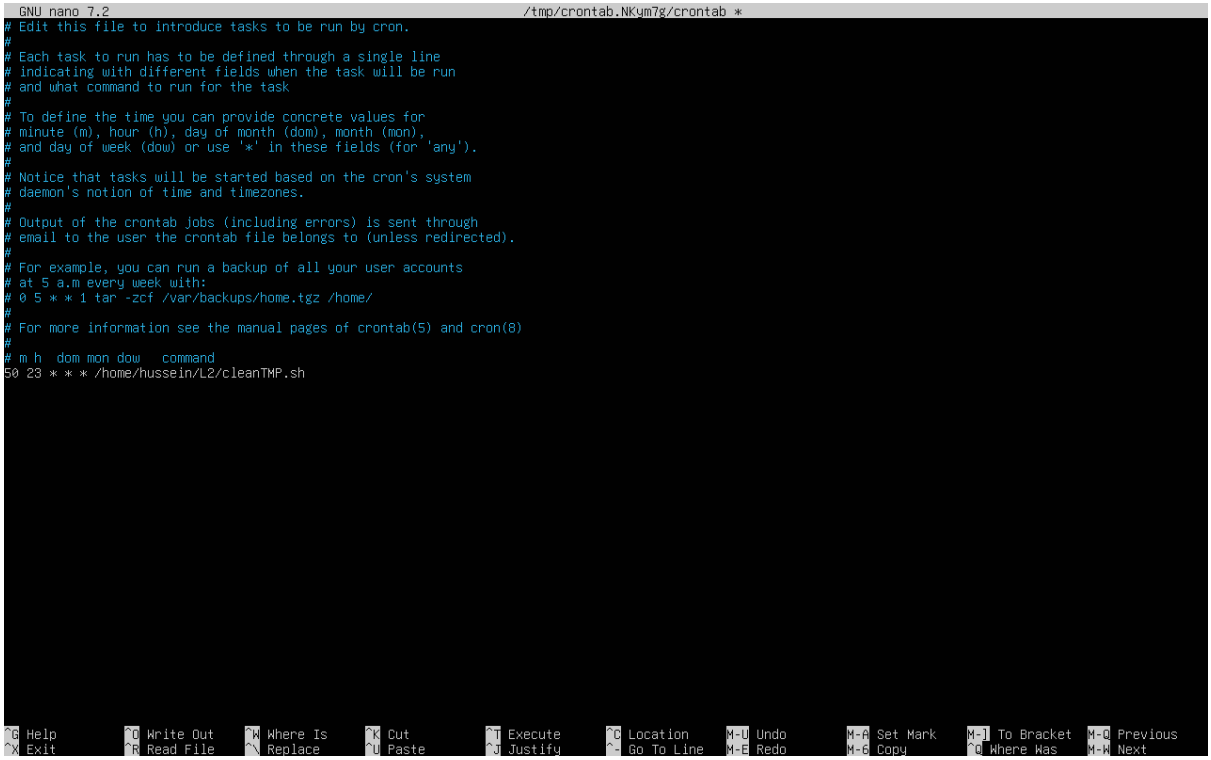
Figur 2: Bilden visar min /tmp mapp samt status för testfilen



Figur 3: Bilden visar min /tmp mapp efter körning av skriptet cleanTMP.sh

I figur 2 ser man att testfilen jag skapade har tagits bort med hjälp av skriptet.

Crontab



Figur 4: Min modifierade crontab

Detta är min modifierade crontab. Första kommandot kör cleanTMP.sh. Syntaxen för crontab: MIN HOUR DOM(day of month) MON(Month) DOW(day of week) CMD.

Jag har satt värden som uppfyller laborationens krav, vid kl 23:50, sedan * för varje dag i månaden, * för varje månad, * för varje dag i veckan.

Skillnaden mellan att köra ett kommando med [at] och med crontab är att [at] kör ett skript en enda gång vid en angiven tidpunkt, medan crontab kör angivna kommandon/skript regelbundna intervaller. Man anger i crontab filen vilken tid, vilka dagar som kommandot ska köras.

System statistik

Vmstat

Processer

r – Antalet processer som körs eller väntar på CPU

b – blockerade processer som väntar på I/O, dessa processer använder inte CPU, men väntar på andra resurser.

Memory

swpd – antalet kB som används i swap (virtuellt minne på disk). Om detta är > 0 betyder det att systemet har bytt ut minne från RAM till disk.

free – Antalet kB lediga i primärminnet.

buff – Minnet som används för buffer (tillfälligt lagringsminne för blockenheter, t.ex disk).

cache – Minnet som används för filcache. Håller ofta filer i minnet för snabb åtkomst, även om de tekniskt sett kan frigöras vid behov.

Swap

Swap används när primärminnet är fullt.

si/swap in – Antalet kB som lästs från swap tillbaka till RAM/s, om > 0 betyder det att systemet måste läsa minne från disken.

so/swap out – Antalet kB som lästs från RAM till swap/s, om > 0 betyder det att systemet har för lite RAM och flyttar minne till disk.

I/O

bi/blocks in – Antalet blocks som lästs från blockenheter/s. (Högt värde = mycket läsning från disk).

bo/blocks out – Antal blocks som skrivits till blockenheter/s. (Högt värde = mycket skrivning till disk).

System

in – Antal interrupts per sekund
cs – Antalt context switches per sekund

CPU

us - % cpu-tid som använts av användarprocesser
sy - % cpu-tid som använts av kärnprocesser.
id - % idle, cpu-tid när cpu går på tomgång
wa - % wait, cpu-tid som väntar på I/O
st - % steal, cpu-tid stulen av hypervisor i virtuell miljö

top

PID – Processens ID

USER – Användaren som startade processen

PR – Prioritetsvärdet

NI – Niceness värdet, desto högre värdet är desto snällare processen är mot andra processer

VIRT – Virtuellt minne, allt minne processen kan använda inkö delat, swap, reserverat.

RES – Faktiskt minne i RAM just nu

SHR – Delat minne med andra processer

S – Processens tillstånd

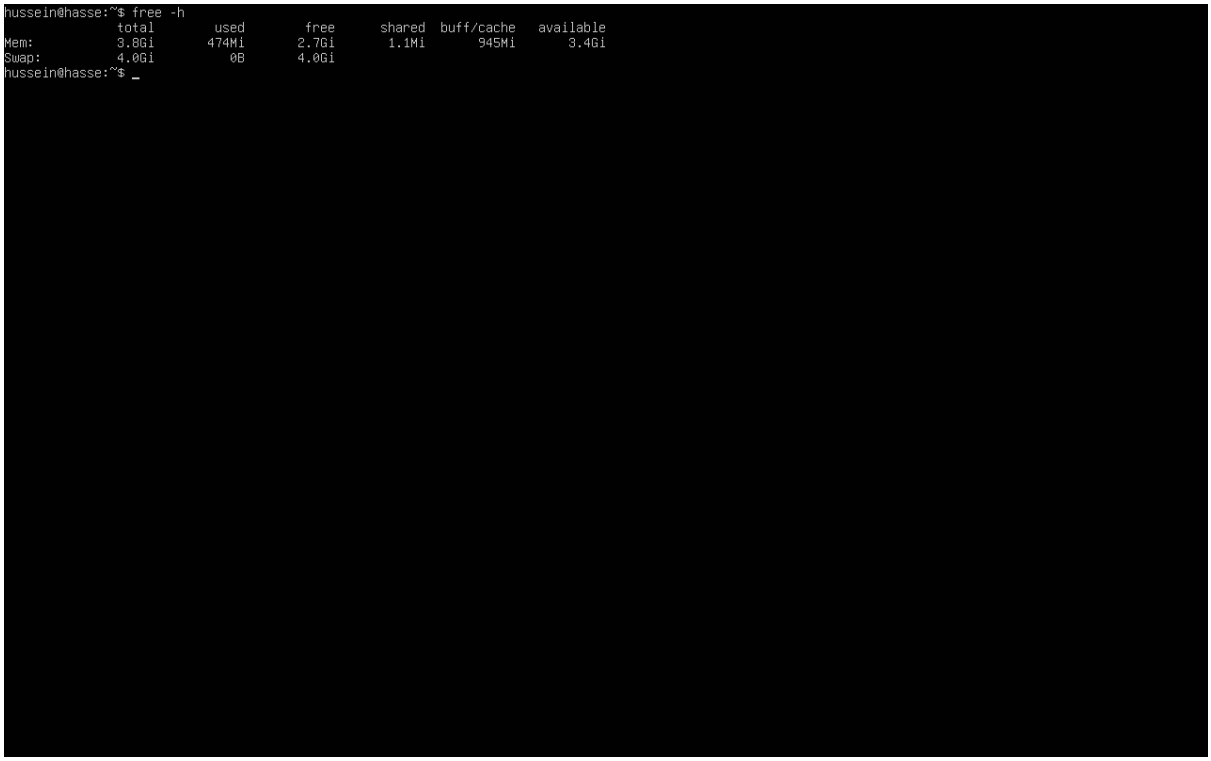
%CPU – Hur mycket CPU tid processen använder

%MEM – Hur mycket minne processen använder

Time+ - Visar hur mycket CPU tid processen har förbrukat sedan start av system

Command – Namnet på processen

Minne & Swap



Figur 5: Kommandot [free -h]

Med kommandot [free -h] visas hur mycket minne och swap som är konfigurerat för detta system.

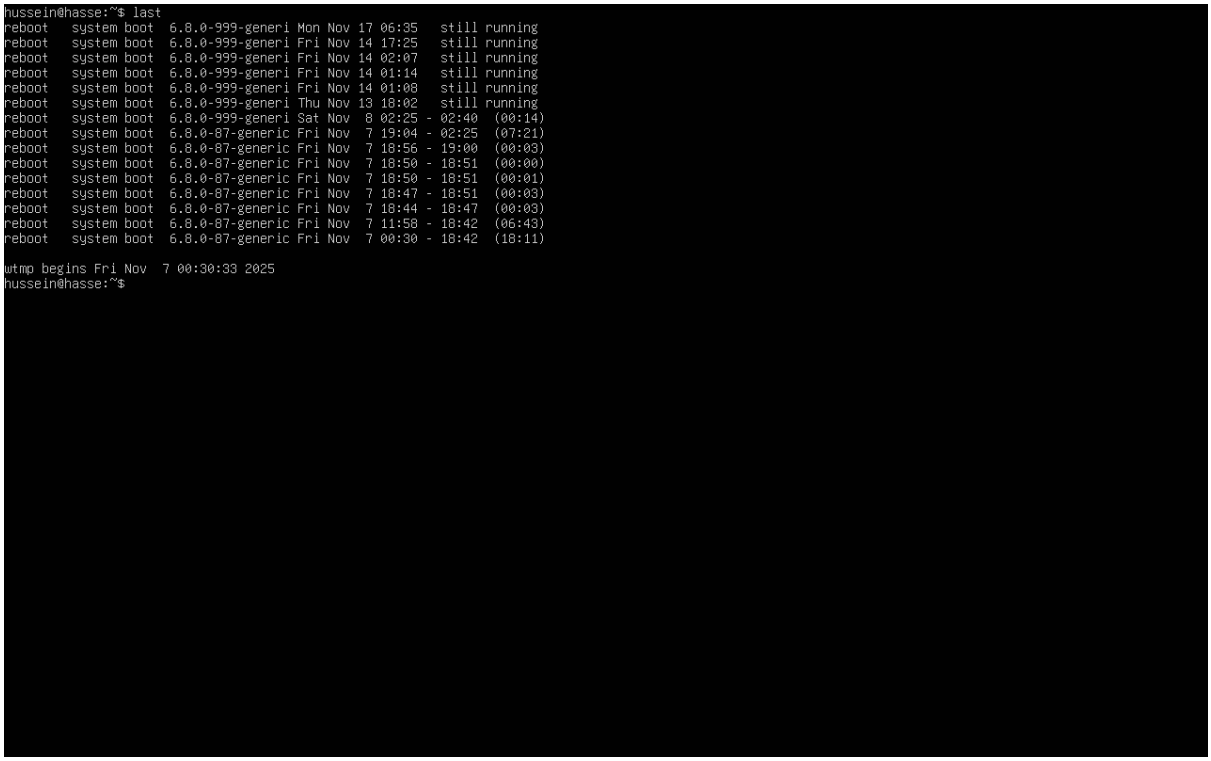
Konfigurerat minne – 3.8GiB

Swap minne – 4GiB

Konfigurerat minne som använts – 474 MiB

Swap minne som använts – 0B

Last



Figur 6: Kommandot [last]

Kommandot [last] hämtar data från /var/log/wtmp. På denna virtuella maskin innehåller wtmp endast poster av typen reboot, alltså systemstarter. Det finns inga registrerade användarinloggar från vanliga användare att redovisa.

W

Kommandot w visar ”hussein” som är inloggad.

Uptime

Den huvudsakliga tanken med kommandot uptime är att visa hur länge systemet har varit igång. Visar också nuvarande tid, antalet användare som är online och load average. Load average är medelvärdet på antalet processer som körs eller väntar på cpu

under de senaste 1, 5, och 15 minuter. Om vi har till exempel ett 4-kärnigt system och första kolumnen visar 2.0, betyder det att systemet körs på 50% kapacitet, om den visar 4, skulle det indikera 100% belastning. Om vi har 4 processer som körs just nu, och 100 processer har väntat under hela senaste 5 minutrarna, kommer andra kolumnen att visa ca 104. Det behöver inte betyda att systemet är i fara utan det visar att schemaläggaren har fullt upp, det kärnorna är upptagna just nu och processerna kommer att köras så de blir schemalagda på exekvering. Load average är alltså köindex som visar hur processkön är belastad.

Skillnader

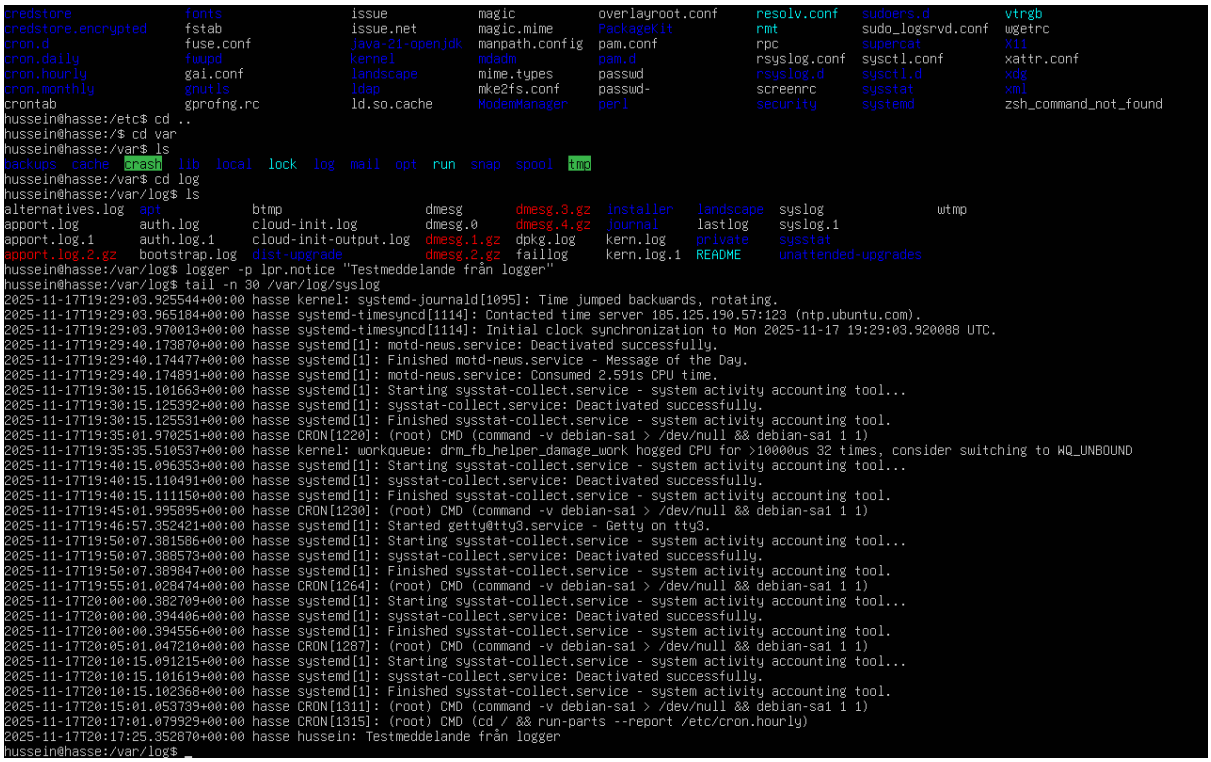
Vmstat ger överblick av systemet, de viktigaste hårdvarorna. Den visar mätvärden för cpu, minne, swap, disk. Den visar inte detaljer om varje process utan den visar en ungefärlig bild över hela systemet. Den visar dem totala värdena där skrivning/läsning till disk, swaps osv.
Top ger djupare bild och mer detaljer om processer. Den är mer specifik för processer och visar vilka processer som tar mest på processor och minne. Den visar också andra kolumner som process id och prioritet, snällhetsgrad.

Logg filer

Dmesg

Kommandot [dmesg] kan ibland kallas för ”driver message”, visar meddelanden direkt från linuxkärnan. Det ger information om systmstart och drivrutiner, t.ex vilken hårdvara som upptäcks och vilka moduler som laddas. Dmesg låter utvecklare att läsa dem i efterhand för felsökning och diagnostik. Om till exempel ett tangentbord inte funkar som det ska eller att den inte känns igen, visas problemet med dmesg, kärnan kan beskriva problemet.

Syslog



Figur 7: Kommandot [tail] som visar senaste 30 raderna i /var/log/syslog

Jag har skickat ett meddelande till denna fil med hjälp av kommandot [logger]. Kommandot är väldigt användbart i skriptar där man kan logga felmeddelanden direkt i skriptet som överlämnas till syslog där utvecklaren kan kika och se vart i koden det gick snett.

Skillnader

Skillnaden mellan syslog och dmesg är att dmesg visar direktmeddelanden från kärnans ringbuffert, den hämtar information om hårdvara, drivrutiner, startup och kärn-händelser, medan syslog innehåller meddelanden från hela systemet. Sysloggen kan också användas på applikationsnivå där man kan till exempel logga tjänter, säkerhet osv. dmesg sitter i primärminnet och rensas vid omstart, medan syslog skrivs över till disk.

Referenser

(u.d.). Hämtat från IBM: https://www.ibm.com/docs/en/db2/12.1.x?topic=tools-about-platform-specific-error-logs

(u.d.). Hämtat från https://www.geeksforgeeks.org/linux-unix/dmesg-command-linux-driver-messages/

(u.d.). Hämtat från https://www.ibm.com/docs/en/aix/7.1.0?topic=commands

Nemeth, E. (2017). *Unix and Linux System Administration Handbook*. Addison-Wesley.